Kevin Kuister

Zusammenfassung: Records

Records

Gefälscht

Datentyp, der aus einzelnen Datentypen zusammengesetzt ist

=> Realisierung als zusammenhängender Speicherbereich, welcher mindestens den benötigten Speicherplatz der einzelnen Komponenten umfasst
↳ Erfolgt via (ggf. rekursiver) Berechnung

=> Adressierung erfolgt über Komponentenadresse beginnend bei 0
↳ Weiter via Addieren der Größe der folgenden Komponenten
Adresse(Komponente) = Basisadresse + Offset(Komponente)

Beispiel

Datum → Tag, Monat, Jahr

```
type Datum is record
  Tag: Byte;      } 1 Byte
  Monat: Byte;    } 1 Byte   } 4 Byte
  Jahr: Short;    } 2 Byte
end Record;
```

Beispiel - Record-Allokation: Heap

Datum-Record, Heap-Pointer hp: 0x1000

1.) Datum allozieren

```
MOVE W hp, R0     ; R0 = 0x1000 (Adr. Datum)
ADD W hp, I4, hp  ; hp = 0x1004 (hp um SP(Datum) erhöhen)
```

2.) Datum initialisieren

```
MOVE B I 15, 0(R0)    ; Tag = 15 (Offset 0, 1 Byte)
MOVE B I 3, 1(R0)     ; Monat = 3 (Offset 1, 1 Byte)
MOVE H I 2024, 2(R0)  ; Jahr = 2024 (Offset 2, 2 Bytes)
```

3.) Zweites Datum allozieren

```
MOVE W hp, R1     ; R1 = 0x1004 (Adr. zweites Datum)
MOVE W hp, I4, hp ; hp = 0x1008
```

Beispiel - Übersetzung Record-Zugriff

Code-Hochsprache:

```
person.Geburtstag.Jahr = 1999;
int alter = 2024 - person.Geburtstag.Jahr;
```

Bekannte Offsets:
Geburtstag: 10
Jahr (in Datum): 2

1) Gesamtoffset: 12

2) Jahr setzen

```
; R5 enthält Basisadr. von Person
MOVE H I 1990, 12(R5) ; Jahr = 1990 (Short = 2 Bytes, Offset: 12)
```

3) Alter berechnen

```
MOVE H 12(R5), R0   ; R0 = 1990
SUB W I 2024, R0, R1 ; => R1 = 34
```